

1. Introduction. A *Hamiltonian cycle* of a graph visits every vertex exactly once and returns to its start. This program counts the Hamiltonian cycles of a graph that are *symmetric*: the graph has the central automorphism that sends each vertex v (numbered 0 to $N - 1$) to its mate $N - 1 - v$, and we want only the cycles that map to themselves under it.

The graph here is the one whose vertices are the squares of an 8×9 board and whose edges are knight moves; it is built by *Board* from the Go port of Knuth's *Stanford GraphBase* (SGB). The program is itself a port to Go, and to GWEB, of Knuth's SGB demonstration program *sham*; the commentary here is newly written.

The search exploits the symmetry directly. Identify every vertex with its mate, giving a *quotient* graph of half the size. A symmetric cycle of the original graph projects onto a *path* in the quotient that runs between two “boundary” vertices, and conversely such a path lifts back to a symmetric cycle. So instead of hunting for cycles in the big graph we hunt for Hamiltonian paths in the half-size one, which is enormously cheaper. Each arc remembers, in the low bit of its length field, whether it crossed between a vertex and its mate's side; those bits decide whether a path lifts to a genuinely symmetric cycle.

```
package main
import (
    "fmt"
    "log"

    gbbasic "github.com/sjnam/go-sgb/gb-basic"
    gbgraph "github.com/sjnam/go-sgb/gb-graph"
)
// We use the GraphBase types directly, under shorter names.
type (
    Graph = gbgraph.Graph
    Vertex = gbgraph.Vertex
    Arc = gbgraph.Arc
)
const (
    mm = 8 // board rows (should be even, so N is even)
    nn = 9 // board columns
)
⟨Subroutines 8⟩
func main() {
    ⟨Build the knight graph and reduce it by symmetry 2⟩
    ⟨Prepare the graph for backtracking and find a minimum-degree vertex 4⟩
    ⟨Check that the search can start 5⟩
    ⟨Search from every pair of edges leaving  $x$  7⟩
    fmt.Printf("Altogether %d solutions and %d wannabees.\n", count, dcount)
}
```

2. Folding the graph in half. *Board* numbers the squares $0, 1, \dots, N-1$, and the central symmetry pairs square v with square $N-1-v$ (its *mate*). Because the vertices live in one contiguous slice, “mate of v ” is just the vertex at index $N-1-v$, and a comparison of vertices is a comparison of their indices.

We walk the lower half of the squares and rewrite their arcs. An arc that stays within a class — its tip’s mate has the *higher* index, so the tip is already the class representative — keeps its tip and is marked with length 0. An arc whose tip belongs to the upper half is redirected to that tip’s mate (the lower-index representative) and marked with length 1, recording that it crosses to the opposite side. Halving $g.N$ then discards the upper half.

This step can create multiple arcs between two classes, and self-loops; we deal with the self-loops next.

⟨Build the knight graph and reduce it by symmetry 2⟩ $\equiv$

```

g, err := gbbasic.Board(mm, nn, 0, 0, 5, 0, false) // piece 5 = knight moves
if err != nil {
    log.Fatal(err)
}
N := g.N
for i := int64(0); N-1-i > i; i++ {
    v := &g.Vertices[i]
    for a := range v.AllArcs() {
        tip := gbgraph.VertexIndex(g, a.Tip)
        u := N - 1 - tip // index of the tip's mate
        if u > tip {
            a.Len = 0
        } else {
            a.Len = 1
            a.Tip = &g.Vertices[u]
        }
    }
}
g.N /= 2

```

This code is used in section 1.

3. Getting ready to search. Self-loops would confuse the forcing rule below (a vertex of apparent degree 1 that is really a dead end), and they are never part of a cycle, so we splice them out of every arc list first.

```

⟨ Remove self-loops 3 ⟩ ≡
  for i := 0; i < n; i++ {
    v := &g.Vertices[i]
    var prev *Arc
    for a := v.Arcs; a ≠ nil; a = a.Next {
      if a.Tip == v {
        if prev ≠ nil {
          prev.Next = a.Next
        } else {
          v.Arcs = a.Next
        }
      } else {
        prev = a
      }
    }
  }

```

This code is used in section 4.

4. The search keeps three pieces of per-vertex bookkeeping, in slices indexed by vertex number: *deg* is the current number of usable arcs (it shrinks as vertices are taken and grows back on backtracking); *taken* marks the vertices already on the path; and *ark* records, by path position, the arc chosen at each step. After clearing the self-loops we count each vertex's degree and remember a vertex *x* of smallest degree to start from — a small fan-out near the start keeps the search tree thin.

```

⟨ Prepare the graph for backtracking and find a minimum-degree vertex 4 ⟩ ≡
  n := int(g.N)
  deg := make([]int64, n)
  taken := make([]bool, n)
  ark := make([]*Arc, n)
  id := func(v *Vertex) int { return int(gbgraph.VertexIndex(g, v)) }
  ⟨ Remove self-loops 3 ⟩
  dmin := n
  var x *Vertex
  for i := 0; i < n; i++ {
    v := &g.Vertices[i]
    taken[i] = false
    d := 0
    for range v.AllArcs() {
      d++
    }
    deg[i] = int64(d)
    if d < dmin {
      dmin = d
      x = v
    }
  }

```

This code is used in section 1.

5. As a consistency check (handy when porting), we print the degree sequence; the same line is printed again at the very end, where it must agree. A start vertex of degree less than 2 cannot lie on any cycle, so there is nothing to do.

⟨ Check that the search can start 5 ⟩ ≡

```

    ⟨ Print the degree sequence 6 ⟩
    if deg[id(x)] < 2 {
        fmt.Printf("The minimum degree is %d (vertex %s)!\n", deg[id(x)], x.Name)
        return
    }

```

This code is used in section 1.

6.

⟨ Print the degree sequence 6 ⟩ ≡

```

    for i := 0; i < n; i++ {
        fmt.Printf(" %d", deg[i])
    }
    fmt.Println()

```

This code is used in section 5.

7. Every Hamiltonian path we want leaves x by one edge and returns to x by another, so we try each unordered pair of edges at x : the first edge starts the path, and the tip of the second is the vertex z where the path must end.

⟨ Search from every pair of edges leaving x 7 ⟩ ≡

```

    count, dcount := 0, 0
    for b := x.Arcs; b.Next ≠ nil; b = b.Next {
        for bb := b.Next; bb ≠ nil; bb = bb.Next {
            findPaths(g, deg, taken, ark, x, bb.Tip, b, &count, &dcount)
        }
    }

```

This code is used in section 1.

8. The backtrack search. *findPaths* enumerates every simple path that begins with arc a , ends at z , visits all $n - 1$ vertices other than x , and never uses x . It is a classic backtracker, written here — as in the original — with explicit labels and **gotos** rather than nested loops, because the control flow (advance, try the next edge, restore state, back up) maps onto labels very directly.

One pruning rule does most of the work: when we step to a vertex v , we lower the recorded degree of each of v 's neighbours by one (they have lost v as a future option). If that makes some still-unused neighbour's degree drop to 1, the path is *forced* to go there next, since it will soon be the only way in or out. If two neighbours become forced at once, this branch is hopeless and we restore and back up. The length field doubles as a tiny stack of flags (+4 marks a forced move, +2 an ordinary one) on top of the crossing bit.

⟨Subroutines 8⟩ ≡

```

func findPaths(g *Graph, deg []int64, taken []bool, ark []*Arc,
  x, z *Vertex, a *Arc, count, dcount *int) {
  ⟨Set up the search state 9⟩
  advance:
    ⟨Take vertex v; record forced moves, or goto backtrack 10⟩
  try:
    ⟨Try arc a and its successors; advance on the first free vertex 11⟩
  restore_all:
    aa = nil
  restore:
    ⟨Undo the degree changes made on entering this level 12⟩
  backtrack:
    ⟨Step back one level and try the next option, or finish 13⟩
  done:
  }

```

This code is also defined in section 15.

This code is used in section 1.

9. *level* is the current path length; the path is complete when it reaches $tmax = n - 1$. We mark x as used, and give the opening arc the forced flag.

⟨Set up the search state 9⟩ ≡

```

n := int(g.N)
tmax := n - 1
id := func(v *Vertex) int { return int(gbgraph.VertexIndex(g, v)) }
var v, u *Vertex
var aa, yy *Arc
var d int64
level := 0
taken[id(x)] = true
a.Len += 4 // the first move is forced

```

This code is used in section 8.

10. On arrival we store the chosen arc, step forward, and mark v taken. If v is the required endpoint z , we have a full path exactly when it is the last vertex and has no other exit; either way this level is finished. Otherwise we lower each neighbour's degree, watching for a forced continuation yy (and bailing out to *restore* if a second forced move appears). A forced move is taken immediately; otherwise we fall through to try v 's arcs in order.

⟨Take vertex v ; record forced moves, or **goto** *backtrack* 10⟩ $\equiv$

```

    ark[level] = a
    level++
    v = a.Tip
    taken[id(v)] = true
    if v == z {
        if level == tmax && deg[id(v)] == 1 {
            ⟨Record a solution 14⟩
        }
        goto backtrack
    }
    yy = nil
    for aa = v.Arcs; aa ≠ nil; aa = aa.Next {
        u = aa.Tip
        d = deg[id(u)] - 1
        if d == 1 && !taken[id(u)] {
            if yy ≠ nil {
                goto restore
            }
            yy = aa
        }
        deg[id(u)] = d
    }
    if yy ≠ nil {
        a = yy
        a.Len += 4
        goto advance
    }
    a = v.Arcs

```

This code is used in section 8.

11. The ordinary case: scan forward for the first arc leading to a vertex not yet taken, flag it, and advance. If none remains, fall through (with aa set to **nil**, so the restore below runs over the whole arc list).

⟨Try arc a and its successors; advance on the first free vertex 11⟩ $\equiv$

```

    for a ≠ nil {
        if !taken[id(a.Tip)] {
            a.Len += 2
            goto advance
        }
        a = a.Next
    }

```

This code is used in section 8.

12. Restoring means giving back the degree we borrowed from the neighbours of the vertex we are leaving, up to the point *aa* where the entry scan stopped.

⟨Undo the degree changes made on entering this level 12⟩ ≡

```

for a = ark[level-1].Tip.Arcs; a ≠ aa; a = a.Next {
    deg[id (a.Tip)]++
}

```

This code is used in section 8.

13. Backing up: drop a level, free that vertex, and strip the move flags from the arc, keeping only its crossing bit. If the move was an ordinary one (no forced flag) we simply try its successor. At the very bottom we are done. A forced move needs care: a forced arc may be one of a pair of parallel arcs, and we must resume from its *partner* in the previous vertex's list rather than backtrack past it; if there is no such twin, the move was truly forced and we restore.

⟨Step back one level and try the next option, or finish 13⟩ ≡

```

level--
a = ark[level]
taken[id (a.Tip)] = false
d = a.Len
a.Len &= 1
if d < 4 {
    a = a.Next
    goto try
}
if level == 0 {
    goto done
}
for aa = ark[level-1].Tip.Arcs; aa ≠ a; aa = aa.Next {
    if aa.Tip == a.Tip {
        aa.Len += 4
        a = aa
        goto advance
    }
}
goto restore_all

```

This code is used in section 8.

14. A completed path lifts to a symmetric cycle only when an odd number of its arcs cross to the mate side; the exclusive-or of the crossing bits tells us which. The others (an even number of crossings) still form a valid path but do not give a new symmetric cycle, so we count them separately as “wannabees.”

⟨Record a solution 14⟩ ≡

```

s := int64(0)
for k := 0; k < tmax; k++ {
    s ^= ark[k].Len & 1
}
if s ≠ 0 {
    *count++
    if *count%100000 == 0 {
        report(g, "", *count, x, ark, tmax)
    }
} else {
    *dcnt++
    if *dcnt%100000 == 0 {
        report(g, ">", *dcnt, x, ark, tmax)
    }
}

```

This code is used in section 10.

15. *report* echoes one tour: the start vertex, then each successive vertex, prefixed by * when the arc into it crosses to the mate side.

⟨Subroutines 8⟩ +≡

```

func report(g *Graph, prefix string, num int, x *Vertex, ark []*Arc, tmax int) {
    fmt.Printf("%s%d: %s", prefix, num, x.Name)
    for k := 0; k < tmax; k++ {
        sep := " "
        if ark[k].Len&1 ≠ 0 {
            sep = "*"
        }
        fmt.Printf("%s%s", sep, ark[k].Tip.Name)
    }
    fmt.Println()
}

```


16. Index.

a: [2](#), [3](#), [8](#), [9](#), [10](#), [11](#), [12](#), [13](#).
aa: [8](#), [9](#), [10](#), [11](#), [12](#), [13](#).
advance: [8](#), [10](#), [11](#), [13](#).
AllArcs: [2](#), [4](#).
Arc: [1](#), [3](#), [4](#), [8](#), [9](#), [15](#).
Arcs: [3](#), [7](#), [10](#), [12](#), [13](#).
ark: [4](#), [7](#), [8](#), [10](#), [12](#), [13](#), [14](#), [15](#).
b: [7](#).
backtrack: [8](#), [10](#).
bb: [7](#).
Board: [1](#), [2](#).
bool: [4](#), [8](#).
count: [1](#), [7](#), [8](#), [14](#).
d: [4](#), [9](#), [10](#), [13](#).
dcount: [1](#), [7](#), [8](#), [14](#).
deg: [4](#), [5](#), [6](#), [7](#), [8](#), [10](#), [12](#).
dmin: [4](#).
done: [8](#), [13](#).
err: [2](#).
false: [2](#), [4](#), [13](#).
Fatal: [2](#).
findPaths: [7](#), [8](#).
fmt: [1](#), [5](#), [6](#), [15](#).
g: [2](#), [3](#), [4](#), [7](#), [8](#), [9](#), [14](#), [15](#).
gbbasic: [1](#), [2](#).
gbgraph: [1](#), [2](#), [4](#), [9](#).
Graph: [1](#), [8](#), [15](#).
i: [2](#), [3](#), [4](#), [6](#).
id: [4](#), [5](#), [9](#), [10](#), [11](#), [12](#), [13](#).
int: [4](#), [8](#), [9](#), [15](#).
int64: [2](#), [4](#), [8](#), [9](#), [14](#).
k: [14](#), [15](#).
Len: [2](#), [9](#), [10](#), [11](#), [13](#), [14](#), [15](#).
level: [9](#), [10](#), [12](#), [13](#).
log: [2](#).
main: [1](#).
make: [4](#).
mate: [2](#).
mm: [1](#), [2](#).
N: [2](#), [4](#), [9](#).
n: [3](#), [4](#), [6](#), [9](#).
Name: [5](#), [15](#).
Next: [3](#), [7](#), [10](#), [11](#), [12](#), [13](#).
nil: [2](#), [3](#), [7](#), [8](#), [10](#), [11](#).
nn: [1](#), [2](#).
num: [15](#).
prefix: [15](#).

prev: [3](#).
Printf: [1](#), [5](#), [6](#), [15](#).
Println: [6](#), [15](#).
report: [14](#), [15](#).
restore: [8](#), [10](#).
restore_all: [8](#), [13](#).
s: [14](#).
sep: [15](#).
string: [15](#).
taken: [4](#), [7](#), [8](#), [9](#), [10](#), [11](#), [13](#).
Tip: [2](#), [3](#), [7](#), [10](#), [11](#), [12](#), [13](#), [15](#).
tip: [2](#).
tmax: [9](#), [10](#), [14](#), [15](#).
true: [9](#), [10](#).
try: [8](#), [13](#).
u: [2](#), [9](#), [10](#).
v: [2](#), [3](#), [4](#), [8](#), [9](#), [10](#).
Vertex: [1](#), [4](#), [8](#), [9](#), [15](#).
VertexIndex: [2](#), [4](#), [9](#).
Vertices: [2](#), [3](#), [4](#).
x: [4](#), [5](#), [7](#), [8](#), [9](#), [14](#), [15](#).
yy: [9](#), [10](#).
z: [7](#), [8](#), [10](#).

- ⟨Build the knight graph and reduce it by symmetry 2⟩ Used in section 1
- ⟨Check that the search can start 5⟩ Used in section 1
- ⟨Prepare the graph for backtracking and find a minimum-degree vertex 4⟩ Used in section 1
- ⟨Print the degree sequence 6⟩ Used in section 5
- ⟨Record a solution 14⟩ Used in section 10
- ⟨Remove self-loops 3⟩ Used in section 4
- ⟨Search from every pair of edges leaving x 7⟩ Used in section 1
- ⟨Set up the search state 9⟩ Used in section 8
- ⟨Step back one level and try the next option, or finish 13⟩ Used in section 8
- ⟨Subroutines 8⟩ Used in section 1
- ⟨Take vertex v ; record forced moves, or **goto** *backtrack* 10⟩ Used in section 8
- ⟨Try arc a and its successors; advance on the first free vertex 11⟩ Used in section 8
- ⟨Undo the degree changes made on entering this level 12⟩ Used in section 8

Symmetric Hamiltonian Cycles

	Section	Page
Introduction	1	1
Folding the graph in half	2	2
Getting ready to search	3	3
The backtrack search	8	5
Index	16	9